

Как устроены нейронные сети?

Османов Осман

5 мая 2026 г.

1 Введение

Нейронные сети лежат в основе многих современных моделей искусственного интеллекта, которые мы используем ежедневно для поиска информации в интернете, для навигации по городу, а также для решения разнообразных прикладных задач с помощью языковых моделей. Однако появилась эта технология ещё в середине XX века.

В 1943 году нейропсихолог Уоррен Мак-Каллок и математик Уолтер Питтс предложили первую математическую модель искусственного нейрона. Эта разработка заложила фундамент современной информатики. В 1957 году физик Фрэнк Розенблатт усовершенствовал идею Мак-Каллока и Питтса, сделав модель более гибкой и добавив концепцию *обучения модели*.

В этом параграфе мы обсудим, с какими ограничениями первых версий нейронных сетей столкнулись математики того времени, а также поговорим про то, как они эволюционировали и стали способны решать весьма сложные задачи.

2 По стопам истории

2.1 Как устроен мозг

Мак-Каллок и Питтс стремились разработать упрощённую математическую модель мозга, поскольку были убеждены, что сетевая архитектура мозга, состоящего из *нейронов и связей между ними*, — это основная причина, по которой человек умеет решать сложные задачи. Рассмотрим, как устроен настоящий нейрон.

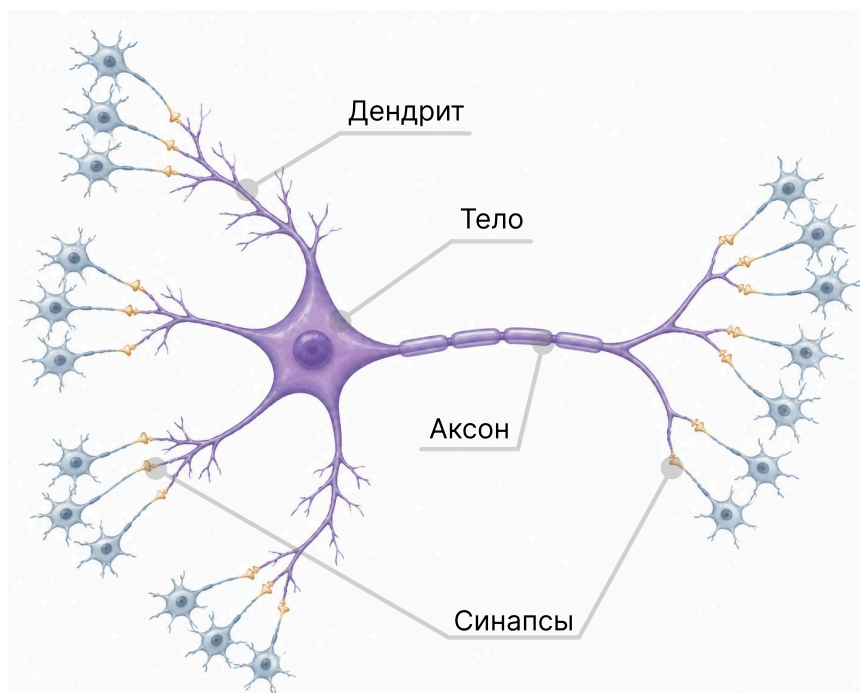


Рис. 1: Устройство нейрона головного мозга, связанного с другими нейронами.

Нейрон головного мозга (см. рис. 1) связывается с другими нейронами через отростки, которые называются *дендритами*. По дендритам электрические сигналы от входных нейронов (синие на рисунке) поступают в *тело нейрона*. Между дендритом и окончанием входного нейрона располагается *синапс*.

Синапс может либо усиливать электрический сигнал, либо наоборот заглушать. Сигналы части входных нейронов усиливаются, а части — заглушаются. Это нужно для того, чтобы более полезные сигналы имели большую интенсивность, а менее полезные — меньшую. Далее в параграфе мы рассмотрим эту концепцию подробнее.

Тело нейрона суммирует сигналы от входных нейронов. Если накопленный сигнал превышает некоторое *пороговое значение*, то нейрон генерирует импульс и передаёт его по *аксонам* другим нейронам. Если же пороговое значение не достигнуто, нейрон молчит и не передаёт сигнал следующим нейронам.

Рассмотрим в качестве примера случай (см. рис. 2), в котором нейрон связан с тремя другими нейронами. Каждый нейрон вносит свой вклад: первый — 10 мВ, второй — 7 мВ, третий — 31 мВ. Сумма входных сигналов — 48 мВ. Предположим, что для данного нейрона установлено пороговое значение в 40 мВ. В таком случае нейрон сгенерирует импульс и передаст его другим нейронам.

Такой принцип возбуждения нейрона при достижении порогового значения называется «*всё или ничего*».



Рис. 2: Иллюстрация принципа «*всё или ничего*».

Нейроны, связываясь и общаясь, формируют обширные сети в головном мозге человека. Такие сети называются *нейронными*. Мак-Каллок и Питтс стремились воссоздать упрощённую математическую модель *нейронной сети*. Рассмотрим их модель подробнее.

2.2 Нейрон Мак-Каллока—Питтса

Мак-Каллок и Питтс хотели формализовать понятие нейрона на языке математической логики. Они решили присуждать нейрону с импульсом значение 1, а молчащему нейрону — значение 0.

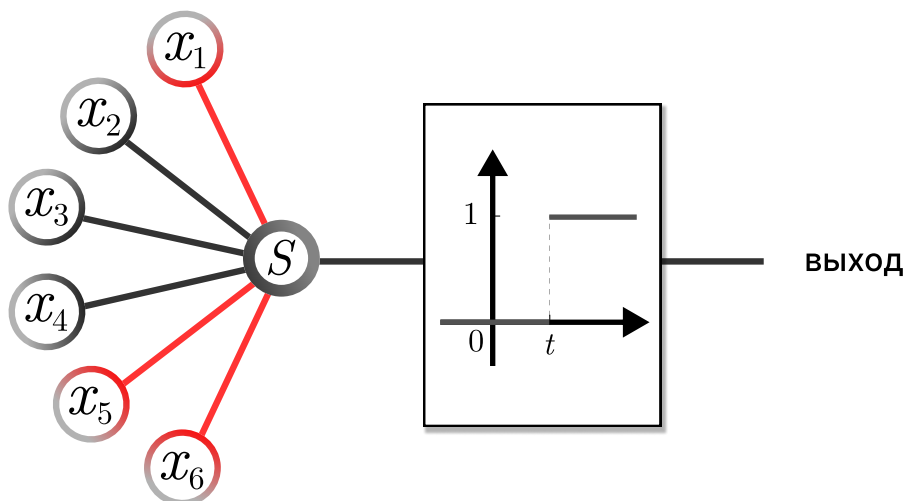


Рис. 3: Модель Мак-Каллока—Питтса.

В модели Мак-Каллока—Питтса (см. рис. 3) связи нейронов подразделяются на два типа: *возбуждающие* (отмечены чёрным) и *тормозящие* (отмечены красным).

Тормозящие нейроны устроены так, что если хотя бы на один из них подать значение 1, то значение основного нейрона, с которым связаны остальные, будет равно 0:

$$S = 0. \quad (1)$$

В качестве примера можно привести микроволновую печь. Возбуждающий нейрон сообщает, дана ли команда запуска печи (нажата ли кнопка), а тормозящий нейрон — открыта ли дверца. Основной нейрон принимает решение, запустить ли печь. Если дверца открыта, то основной нейрон примет значение 0, и печь не заработает.

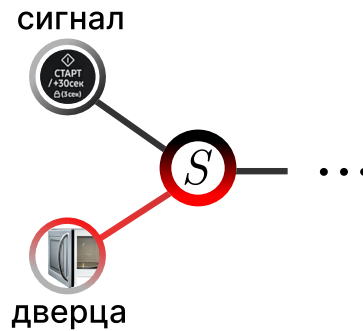


Рис. 4: Иллюстрация к примеру.

Если же ни один тормозящий нейрон не даёт импульса, то в основном нейроне суммируются значения возбуждающих нейронов:

$$S = x_2 + x_3 + x_4. \quad (2)$$

Таким образом,

$$S = \begin{cases} 0, & (1 - x_1)(1 - x_5)(1 - x_6) = 0 \\ x_2 + x_3 + x_4, & \text{иначе.} \end{cases} \quad (3)$$

Это можно записать так:

$$S = (x_2 + x_3 + x_4)(1 - x_1)(1 - x_5)(1 - x_6). \quad (4)$$

После того, как сформировалось значение S , нужно проверить, достигает ли оно порога t . Если $S \geq t$, то нейрон подаст на выход 1, а если $S < t$, то 0. Для этого введём функцию $f(S)$ (см. рис. 5):

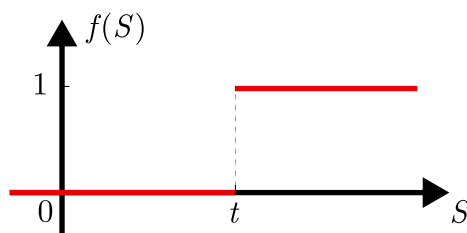


Рис. 5: График пороговой функции.

порогом активации.

Определим функцию $f(S)$ как кусочно-заданную:

$$f(S) = \begin{cases} 1, & S \geq t \\ 0, & S < t. \end{cases} \quad (5)$$

Такая функция называется *пороговой функцией активации*, а число t — *порогом активации*.

Выход нейрона после обработки пороговой функцией называют *активацией*. Отсюда и название «*порог активации*». Таким образом, активацию можно записать в следующем виде:

$$f((x_2 + x_3 + x_4)(1 - x_1)(1 - x_5)(1 - x_6)). \quad (6)$$

В биологической модели, если бы полученное выражение равнялось 1, то нейрон подал бы импульс другим нейронам, а если 0, то нейрон бы молчал. Однако в модели Мак-Каллока—Питтса нейрон не молчит, а всегда подаёт сигнал: либо 0, либо 1.

Мы успешно реализовали модель нейрона. Обозначим его активацию за A и обобщим модель до случая, когда на вход подаются значения $x_1, \dots, x_n$. Таким образом, A — это функция от переменных $x_1, \dots, x_n$. В общем виде можно записать активацию A нейрона следующим образом:

$$A(x_1, \dots, x_n) = f\left(\left(\sum_{i \in E} x_i\right) \left(\prod_{i \in I} (1 - x_i)\right)\right); \quad E \cup I = [1, n]. \quad (7)$$

где E — множество номеров возбуждающих нейронов (от английского слова *excitatory* — *возбуждающий*), а I — тормозящих (*inhibitory* — *тормозящий*).

Теперь, если мы хотим, чтобы функция A принимала значения в соответствии с некоторой таблицей истинности, нам нужно правильно подобрать параметры E , I и t , где E и I — множества, а t — целое число. Однако для решения задачи мы можем воспользоваться лишь переборным алгоритмом, который произведёт полный перебор всех возможных параметров E , I и t , то есть функций A . Никакого алгоритма, который автоматически находил бы E , I и t быстрее полного перебора, предложено не было.¹

Для n входных нейронов существует $\mathcal{O}(n \cdot 2^n)$ вариантов функции A . В современных задачах количество входов может исчисляться тысячами. Если $n \sim 10^3$, то перебор всех возможных функций займёт немыслимые $\approx 10^3 \cdot 2^{10^3}$ операций. Это число примерно в 10^{221} раз превышает число атомов в наблюдаемой вселенной.

x_1	x_2	$x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

(a) XOR

x_1	x_2	$x_1 \& x_2$
0	0	0
0	1	0
1	0	0
1	1	1

(b) И

x_1	x_2	$x_1 \parallel x_2$
0	0	0
0	1	1
1	0	1
1	1	1

(c) ИЛИ

Рис. 6: Таблицы истинности для логических операций XOR, И, ИЛИ.

2.3 Проблема XOR

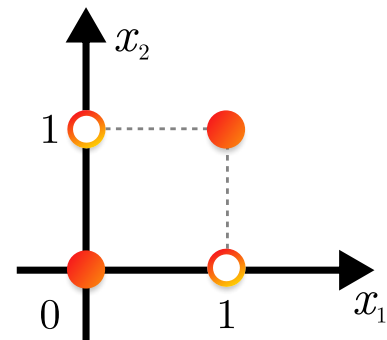
Из курса информатики мы знаем, как устроена бинарная операция XOR (*исключающее ИЛИ*). Таблица истинности для XOR изображена на рис. 6 (a).

Обратите внимание на то, что результат применения операции равен 1 только в тех случаях, когда операнды разные, а в остальных случаях — 0.

Для построения логической функции XOR на основе нейрона Мак-Каллока—Питтса нужно подобрать оптимальные параметры E , I и t . Из таблицы истинности заметим, что ни один из нейронов x_1 и x_2 не является тормозящим, иначе на второй или третьей строчке таблицы истинности выходом модели было бы 0. Таким образом, оба нейрона x_1 и x_2 являются возбуждающими.

Вспомним, что модель складывает значения возбуждающих нейронов, а затем проверяет, достигла ли сумма порога:

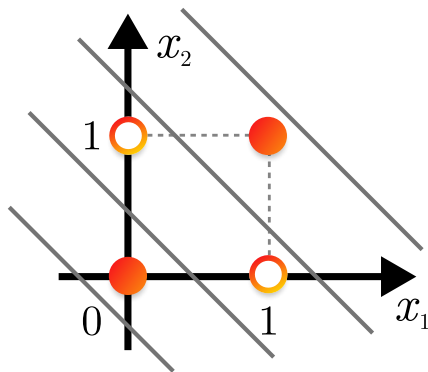
$$A(x_1, x_2) = \begin{cases} 1, & x_1 + x_2 \geq t \\ 0, & x_1 + x_2 < t. \end{cases} \quad (8)$$



Для удобства в системе координат Ox_1x_2 расположим в точках с координатами (0; 1) и (1; 0) закрашенные кружочки (выход модели — 1), а в точках с координатами (0; 0) и (1; 1) — незакрашенные (выход модели — 0).

Рис. 7: Точки на плоскости.

¹Стоит отметить, что Мак-Каллок и Питтс не ставили своей целью изобрести алгоритм, который подбирал бы *оптимальные параметры*. Их задача заключалась лишь в том, чтобы построить формальную модель нейрона.



Если мы перепишем условия относительно x_2 , то получим:

$$A(x_1, x_2) = \begin{cases} 1, & x_2 \geq t - x_1 \\ 0, & x_2 < t - x_1. \end{cases} \quad (9)$$

Из уравнения (9) следует, что достаточно провести прямую вида

$$x_2 = t - x_1 \quad (10)$$

Рис. 8: Иллюстрация проблемы XOR.

в системе координат Ox_1x_2 .

В заданной модели положение прямой задаётся однозначно параметром t . Точки, которые окажутся выше прямой, модель обозначит как 1, а точки, которые окажутся ниже — как 0.

Однако из рис. 8 видно, что какую бы прямую мы не попытались провести, сделать так, чтобы по одну сторону от прямой располагались *только* закрашенные, а по другую — *только* незакрашенные кружочки, *невозможно*.

Это ограничение получило название «*проблема XOR*». Эту проблеме в 1969 году учёные Марвин Минский и Сеймур Пейперт описали в книге «*Перцептроны*». Для решения подобных сложных для одного нейрона задач нужно использовать более комплексную архитектуру, о которой мы поговорим далее.

Если вам интересно оценить пределы возможностей нейрона МакКаллока—Питтса, то это можно сделать в этом ноутбуке.

2.4 Решение проблемы XOR с помощью нейронной сети

Решение проблемы XOR заключается в использовании нескольких нейронов вместо одного. На рис. 9 наглядно изображена схема. Рассмотрим её подробнее.

Нейрон s_1 принимает на вход два значения: x_1 и x_2 , причём вход от нейрона x_2 установлен тормозящим. Теперь, если x_2 примет значение 1, активация нейрона s_1 (то есть его выход) будет равна нулю, и он не подаст импульс. Обратите внимание на то, что для нейрона s_1 установлен порог $t = 1$, а значит, s_1 подаст импульс только в случае, если $x_1 = 1$.

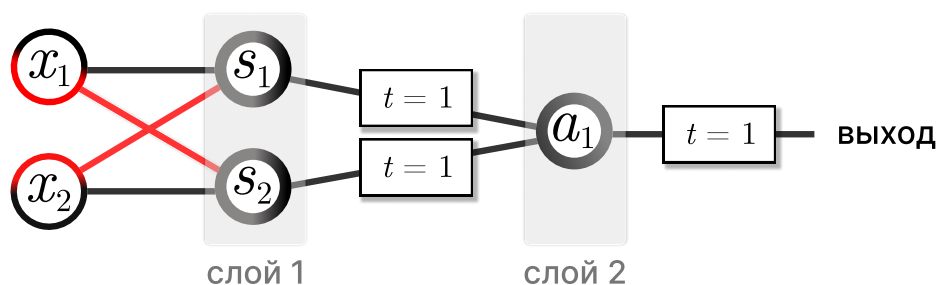


Рис. 9: Решение проблемы XOR с помощью двухслойной модели.

Таким образом, выход нейрона s_1 будет равен 1 только в случае, если $x_1 = 1$ и $x_2 = 0$. Это один из возможных вариантов для *исключающего ИЛИ*. Аналогично построен нейрон s_2 , однако он принимает значение 1 только в случае, если $x_1 = 0$ и $x_2 = 1$.

Набор из нейронов s_1 и s_2 образует первый *слой нейронной сети*. Второй слой состоит всего из одного нейрона — a_1 . Он принимает в качестве входных значений активации нейронов *предыдущего* слоя, как показано на рис. 9.

Нейрон a_1 подаёт импульс (то есть $a_1 = 1$) только в случае, если либо $s_1 = 1$, либо $s_2 = 1$. Иначе говоря, выход нейрона a_1 равен 1 только в случае, если пара (x_1, x_2) равна $(0, 1)$ или $(1, 0)$, и 0 в остальных случаях, полностью повторяя поведение *исключающего ИЛИ*.

Ранее мы говорили, что проблема *исключающего ИЛИ* заключается в линейной неразделимости классов. Здесь же мы проводим две прямые и помечаем область, в которую должна попасть входная точка, чтобы модель дала на выходе 1 (см. рис. 10).

Таким образом, сложная задача разбилась на более простые. Чем больше слоёв в модели, тем более сложные границы между классами она может выстраивать.

Мак-Каллок и Питтс доказали, что с помощью *двухслойной* нейронной сети можно построить любую логическую функцию. Однако мы уже обсуждали, что подбор параметров для каждого нейрона — это очень трудоёмкий и сложный процесс. Отсутствие эффективных способов подбора параметров стало одной из причин так называемой *зимы искусственного интеллекта* — периода замедления развития ИИ.

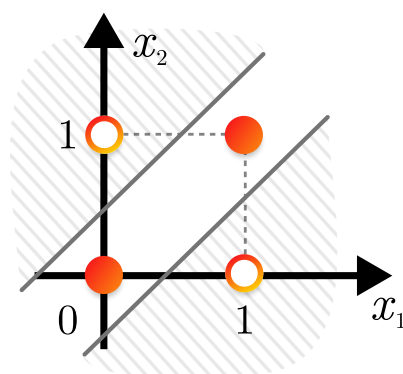


Рис. 10: Иллюстрация решения проблемы XOR.

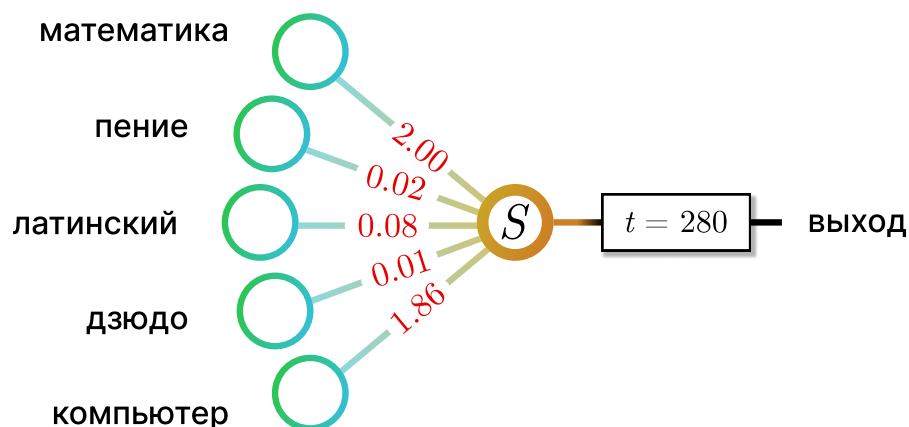


Рис. 11: Иллюстрация к задаче о поступлении в университет.

2.5 Современные нейронные сети

В современных нейронных сетях за основу взята модель, предложенная учёным Фрэнком Розенблаттом в 1957 году. Главное её отличие от модели Мак-Каллока—Питтса заключается в том, что на вход поступают не бинарные значения (0 или 1), а любые вещественные числа. Это позволяет передавать интенсивность сигналов точнее.

Также Розенблатт отказался от концепции тормозящих и возбуждающих связей нейронов. В его модели значимость каждого входного сигнала определяется его *весом*. *Весом* называется числовой коэффициент, на который умножается значение входного сигнала. Чем больше абсолютное значение веса, тем сильнее данный сигнал влияет на результат работы нейрона.

Приведём в качестве примера задачу. Университеты по всей стране хотят построить нейронную сеть, которая определяла бы шансы поступления абитуриента в их вуз. Для каждого абитуриента известны его способности в:

- математике (x_{math}),
- пении (x_{sing}),
- знании латинского языка (x_{latin}),
- дзюдо (x_{judo}),
- умении работать с компьютером (x_{comp}).

Все параметры задаются от 0 до 100. С помощью весов мы можем показать модели, какие способности для поступления более важны, а какие — менее.

Рассмотрим случай с техническим университетом (см. рис. 11). Веса будем задавать в пределах от 0 до 2. Способности в математике гораздо важнее способностей в пении для поступления в технический университет, поэтому пению мы присвоим небольшой вес ($w_{\text{sing}} = 0.02$), а математике — большой ($w_{\text{math}} = 2.00$).

Теперь, если у абитуриента способности в пении на высоте (например, $x_{\text{sing}} = 100$), то на поступление это не сильно повлияет, поскольку вес у способности петь небольшой: $100 \cdot 0.02 = 2$.

Вместе с тем, если у абитуриента отличные способности в математике (например, $x_{\text{math}} = 90$), то этот факт окажет существенное влияние на решение приёмной комиссии: $90 \cdot 2.00 = 180$.

Аналогичным образом необходимо умножить способность студента в каждой сфере на соответствующий вес, а затем получившиеся числа сложить:

$$S = w_{\text{math}}x_{\text{math}} + w_{\text{sing}}x_{\text{sing}} + w_{\text{latin}}x_{\text{latin}} + w_{\text{judo}}x_{\text{judo}} + w_{\text{comp}}x_{\text{comp}}. \quad (11)$$

Такая сумма, в которой перед сложением происходит умножение на веса, называется *взвешенной*. Если взвешенная сумма S окажется больше, чем 280, то университет принимает абитуриента. Оцените ваши шансы на поступление в этот университет (поступить в него не так уж и сложно).

Набор весов у театрального университета сильно бы отличался. Навыки пения для театрального университета бесспорно значимее математических способностей.

Идея взвешенных сумм сделала модель нейрона более гибкой. Если ранее мы подбирали множества E и I , то теперь мы должны подбирать наборы весов. Чем точнее мы подберём веса, тем лучше будет работать модель. Процесс автоматического подбора весов называется *обучением модели*, и занимается этим целая наука — раздел искусственного интеллекта — *машинное обучение*.

Веса нейрона можно записать в вектор $\mathbf{w}$:

$$\mathbf{w} = (w_{\text{math}}, w_{\text{sing}}, w_{\text{latin}}, w_{\text{judo}}, w_{\text{comp}}), \quad (12)$$

а входные значения — в вектор $\mathbf{x}$:

$$\mathbf{x} = (x_{\text{math}}, x_{\text{sing}}, x_{\text{latin}}, x_{\text{judo}}, x_{\text{comp}}), \quad (13)$$

тогда взвешенную сумму S можно записать в виде скалярного произведения:

$$S = \mathbf{w} \cdot \mathbf{x}. \quad (14)$$

Для получения окончательного выхода нейрона — активацию — нужно применить пороговую функцию:

$$A(\mathbf{x}) = f(S) = f(\mathbf{w} \cdot \mathbf{x}). \quad (15)$$

Таким образом, вся модель — это просто функция $A(\mathbf{x})$. Вид этой функции зависит от вектора весов $\mathbf{w}$, а также от порога t , что часто записывают так (слева от черты аргумент, а справа — параметры):

$$A(\mathbf{x}|\mathbf{w}, t) = f(\mathbf{w} \cdot \mathbf{x}). \quad (16)$$

В нейронных сетях у каждого нейрона есть свой вектор весов $\mathbf{w}$. Вектор весов нейрона скалярно перемножается с вектором выхода предыдущего слоя (или вектором входа, если предыдущего слоя нет).

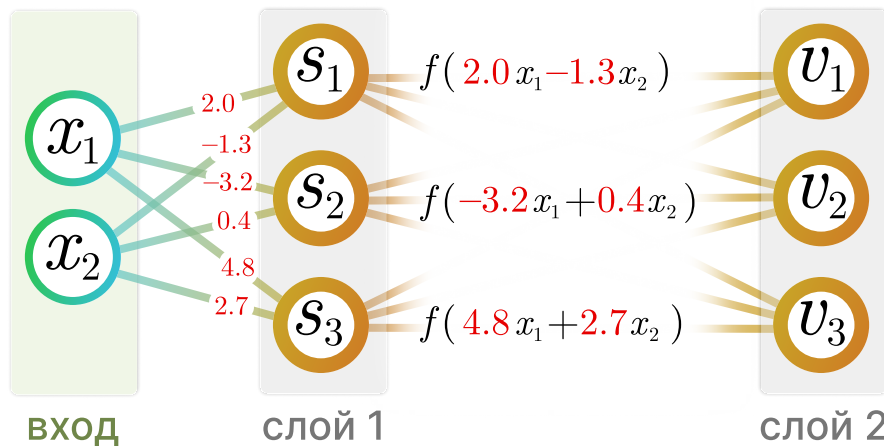


Рис. 12: Модель нейронной сети.

На рис. 12 изображена модель нейронной сети. Обозначим вектор весов для нейрона s_1 за $\mathbf{w}_1$, s_2 — за $\mathbf{w}_2$ и т.д. Вектор предыдущего слоя (в данном случае вектор входа) обозначим за $\mathbf{x}$:

$$\mathbf{x} = (x_1, x_2). \quad (17)$$

Теперь активацию нейрона s_1 можно записать через скалярное произведение:

$$s_1 = f(\mathbf{w}_1 \cdot \mathbf{x}). \quad (18)$$

Аналогичным образом можно записать активации и для оставшихся нейронов: $s_2 = f(\mathbf{w}_2 \cdot \mathbf{x})$ и $s_3 = f(\mathbf{w}_3 \cdot \mathbf{x})$. Теперь, чтобы посчитать активации следующего слоя (слоя 2), нужно построить вектор активаций текущего слоя (слоя 1):

$$\mathbf{s} = (s_1, s_2, s_3) = (f(\mathbf{w}_1 \cdot \mathbf{x}), f(\mathbf{w}_2 \cdot \mathbf{x}), f(\mathbf{w}_3 \cdot \mathbf{x})). \quad (19)$$

Обычно записывают вектор $\mathbf{s}$ следующим образом:

$$\mathbf{s} = f(\mathbf{w}_1 \cdot \mathbf{x}, \mathbf{w}_2 \cdot \mathbf{x}, \mathbf{w}_3 \cdot \mathbf{x}), \quad (20)$$

то есть вместо того, чтобы независимо считать активации для каждого нейрона, сначала считают вектор скалярных произведений:

$$\mathbf{h} = (\mathbf{w}_1 \cdot \mathbf{x}, \mathbf{w}_2 \cdot \mathbf{x}, \mathbf{w}_3 \cdot \mathbf{x}), \quad (21)$$

а затем к этому вектору применяют пороговую функцию:

$$\mathbf{s} = f(\mathbf{h}). \quad (22)$$

Для этого пороговую функцию *векторизуют* — реализовывают так, чтобы пороговая функция автоматически применилась ко всем элементам вектора:

$$f(a, b, c) \mapsto (f(a), f(b), f(c)). \quad (23)$$

Векторизация позволяет значительно ускорить вычисления. Заметим, что $\mathbf{h}$ — это вектор скалярных произведений, а значит, можно записать *матрицу весов* W_1 (для слоя 1) следующим образом:

$$W_1 = \begin{bmatrix} - & \mathbf{w}_1 & - \\ - & \mathbf{w}_2 & - \\ - & \mathbf{w}_3 & - \end{bmatrix} \quad (24)$$

тогда

$$\mathbf{s} = f(W_1 \mathbf{x}). \quad (25)$$

Вот и всё: мы записали прямую зависимость между $\mathbf{s}$ и $\mathbf{x}$, то есть между предыдущим и текущим слоями. Такое преобразование позволяет сильно ускорить вычисления активаций.

Для примера на рис. 12 формула бы выглядела так:

$$\mathbf{s} = f \left(\begin{bmatrix} 2.0 & -1.3 \\ -3.2 & 0.4 \\ 4.8 & 2.7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right). \quad (26)$$

Теперь процесс *обучения* модели заключается в поиске *оптимальной* матрицы W_1 и порога t . Заметим существенную проблему: в нашей модели мы не можем для каждого нейрона установить свой порог. Если мы установим какой-нибудь порог, он применится ко всем.

Пусть для пороговой функции установлен порог t . Тогда запишем функцию $f(x)$:

$$f(x) = \begin{cases} 1, & x \geq t \\ 0, & x < t. \end{cases} \quad (27)$$

Однако такая запись равносильна следующей:

$$f(x) = \begin{cases} 1, & x - t \geq 0 \\ 0, & x - t < 0. \end{cases} \quad (28)$$

Пусть $\sigma(x)$ — пороговая функция с порогом 0 (см. рис. 13). Тогда запись $f(x)$ равносильна записи $\sigma(x - t)$. Таким образом, мы можем для каждого нейрона установить собственный порог следующим образом:

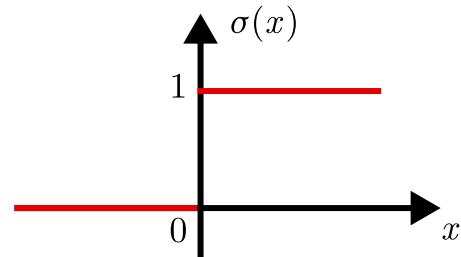


Рис. 13: График пороговой функции с порогом 0.

$$\mathbf{s} = \sigma \left(\begin{bmatrix} 2.0 & -1.3 \\ -3.2 & 0.4 \\ 4.8 & 2.7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} -t_1 \\ -t_2 \\ -t_3 \end{bmatrix} \right), \quad (29)$$

где t_i — порог для i -го нейрона.

Прибавленный вектор констант называется *вектором смещений*. Смещение обозначим символом $\mathbf{b}_1$ (от английского слова *bias* — «смещение»; индекс указывает на номер слоя — первый). Таким образом, выход слоя можно найти по следующей формуле:

$$\mathbf{s} = f(W_1 \mathbf{x} + \mathbf{b}_1), \quad (30)$$

где

- W_1 — матрица весов первого слоя,
- $\mathbf{b}_1$ — вектор смещений первого слоя,
- $\mathbf{x}$ — выход предыдущего слоя (или вход модели),
- $f(x)$ — пороговая функция с порогом 0.

Аналогичным образом запишем выход слоя 2:

$$\mathbf{v} = f(W_2 \mathbf{s} + \mathbf{b}_2). \quad (31)$$

Вот мы и построили шаг за шагом прототип *персептрона* — многослойной нейронной сети.

2.6 Персептрон

Построенная нами ранее модель уже достаточно мощна, однако у неё есть очень существенный недостаток: не было предложено эффективного метода обучения такой модели.

В 1949 году нейропсихолог Дональд Хебб сформулировал принцип, согласно которому связь между двумя нейронами, которые часто срабатывают вместе, усиливается.

Розенблатту успешно удалось спроецировать биологическую идею Хебба на свою модель. Он построил алгоритм обучения *однослойной* сети, но не смог обобщить до многослойной.

В 1958 году Розенблатт представил миру **MARK I Perceptron** (см. рис. 14) — первый в мире *нейрокомпьютер* — устройство, которое работает на основе нейронных сетей.

MARK I Perceptron представляет собой огромный шкаф с большим количеством проводов. Он обучен распознавать некоторые английские буквы.

На рис. 14 виден расположенный перед карточкой с буквой оптический сканер со светочувствительным слоем с 400 фотоэлементами (20×20). Затем полученные данные отправляются в большую физически построенную нейронную сеть, ответ от которой высвечивается с помощью индикатора.

Нейрокомпьютер вызвал сенсацию и оказал большое влияние на развитие нейронных сетей. Однако, как мы уже говорили, в 1969 году учёный Марвин Минский выявил существенные ограничения у однослойных нейронных сетей (например, проблема *исключающего ИЛИ*), которые и использовались в **MARK I Perceptron**.

Но почему Розенблатт не применял многослойные сети, если они решали все проблемы однослойных? Дело, как мы уже говорили, в том, что способов обучать многослойные сети не было. Это и стало причиной урезанных вложений в исследования искусственного интеллекта и *зимы искусственного интеллекта*.

В 1986 году проблема была решена учёными Дэвидом Румельхартом, Джеффри Хинтоном и Рональдом Уильямсом. О том, какую идею

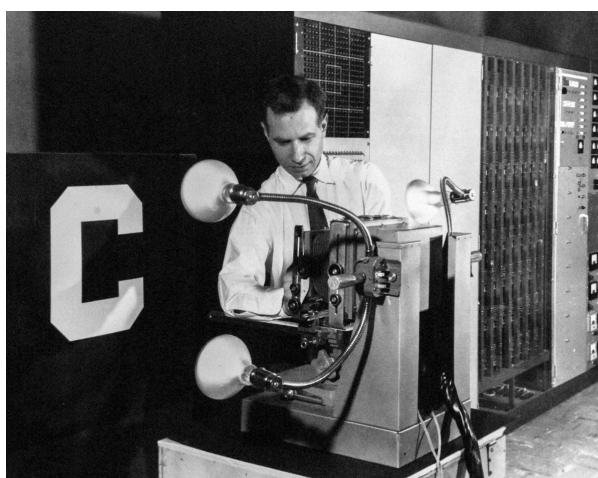


Рис. 14: Фрэнк Розенблатт и **MARK I Perceptron**.

они предложили, и об обучении моделей в целом мы поговорим в главе ?.

2.7 Теорема Цыбенко

Ранее мы утверждали, что с помощью модели двухслойной нейронной сети Мак-Каллока—Питтса можно построить любую логическую функцию. Однако мы не всегда работаем с логическими функциями. Можно ли этот результат обобщить до непрерывных функций?

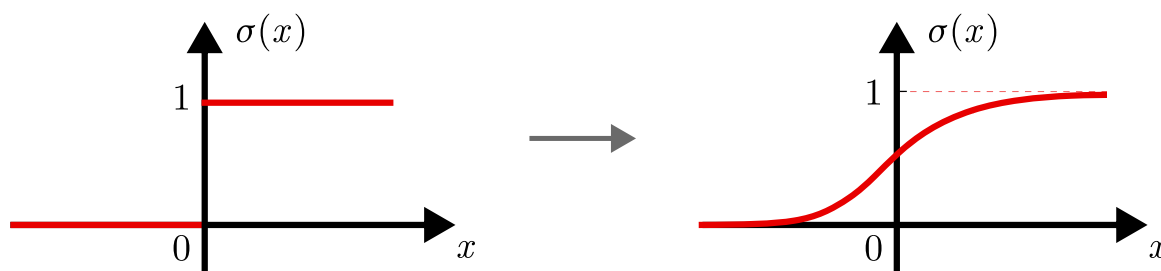


Рис. 15: Переход от пороговой функции к сигмоидальной.

Когда мы работали с нейронами Мак-Каллока—Питтса и Розенблатта, для получения активации мы применяли пороговую функцию активации. Однако методов обучения для моделей с пороговой функцией, как мы уже говорили, нет.

В 1986 году учёные Дэвид Румельхарт, Джеффри Хинтон и Рональд Уильямс, которых мы вскользь затрагивали, предложили использовать *непрерывную* функцию активации, которая легко дифференцируется и в целом повторяет поведение пороговой функции.

Они выбрали *сигмоидальную* функцию:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (32)$$

На рис. 15 изображён график пороговой функции (слева) и сигмоидальной (справа).

В 1989 году математик Джордж Цыбенко доказал, что если на первом слое нейронной сети использовать сигмоидальную функцию активации, а на втором слое — линейную ($f(x) = x$, что эквивалентно отсутствию функции активации) (см. рис. 16), то такая нейронная сеть сможет приблизить любую непрерывную функцию на конечном интервале с любой точностью.

Предположим, что существует некоторая функция f , такая что

$$f : \mathbb{R}^m \rightarrow \mathbb{R}^n, \quad (33)$$

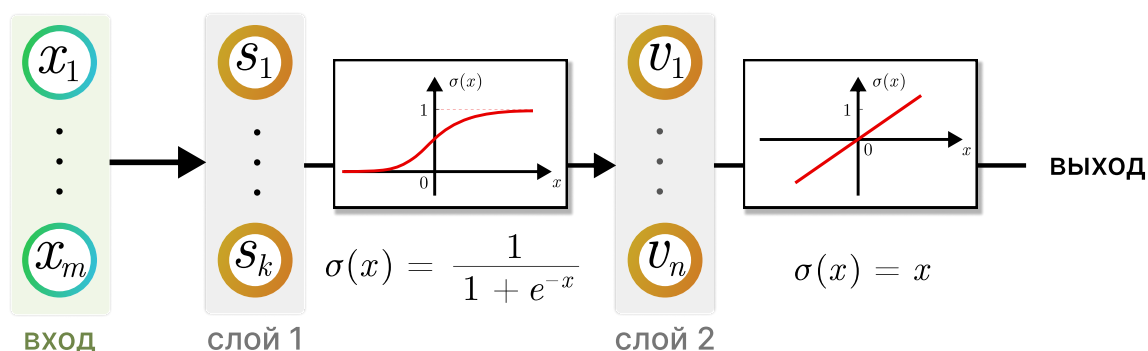


Рис. 16: Архитектура, предложенная Джорджем Цыбенко.

то есть принимающая на вход m аргументов и возвращающая вектор из n чисел. Тогда мы можем построить нейронную сеть, представленную на рис. 16.

Первый слой нейронной сети содержит k нейронов. Чем больше нейронов в первом слое, тем точнее нейронная сеть сможет приблизить функцию f . Второй слой содержит n нейронов, которые формируют вывод.

Теперь, если мы подадим на вход модели некоторый набор аргументов $x_1, \dots, x_m$, мы будем ожидать, что выход модели $(v_1, \dots, v_n)$ будет очень близким к значению функции $f(x_1, \dots, x_m)$. Самая сложная задача заключается в подборе подходящих весов.

Теорема Цыбенко оказала очень большое влияние на развитие нейронных сетей. Она дала строгое математическое доказательство тому, что нейросети являются *универсальными аппроксиматорами*.

Если в случае с моделью Мак-Каллока—Питтса и задачей XOR одна прямая не могла разделить точки на два класса, то архитектура с одним скрытым слоем и сигмоидой, обоснованная Цыбенко, позволяет проводить нелинейные границы и приближать гораздо более сложные функции.

В этом ноутбуке вы можете протестировать теорему Цыбенко, а также проверить качество приближения при разных значениях k .

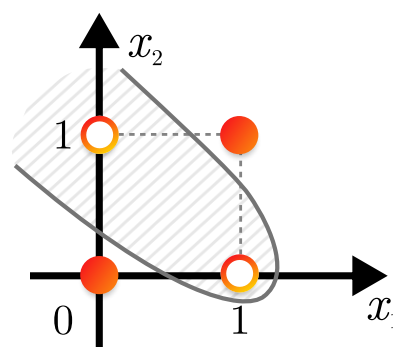


Рис. 17: Иллюстрация теоремы Цыбенко.

3 Нейронные сети на практике

Одна из задач, которую можно решить с помощью нейронных сетей, — это задача *распознавания изображений*. Мы рассмотрим простой случай — распознавание рукописных цифр.

В 1994 году Национальный институт стандартов и технологий США (*National Institute of Standards and Technology*) опубликовал набор обучающих данных (датасет) **MNIST** из 70 000 изображений рукописных цифр.

Каждое изображение (см. рис. 18) имеет разрешение $28 \times 28 = 784$ пикселей. Таким образом, каждое изображение — это вектор из 784 чисел, описывающих яркость каждого пикселя.

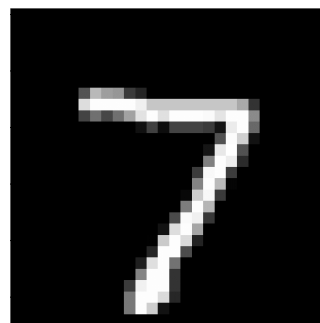


Рис. 18: Пример изображения из *MNIST*.

Мы можем построить нейронную сеть, которая принимает на вход 784 числа и возвращает 10 чисел — *распределение вероятностей*. Подробнее рассмотрим архитектуру на рис. 19.

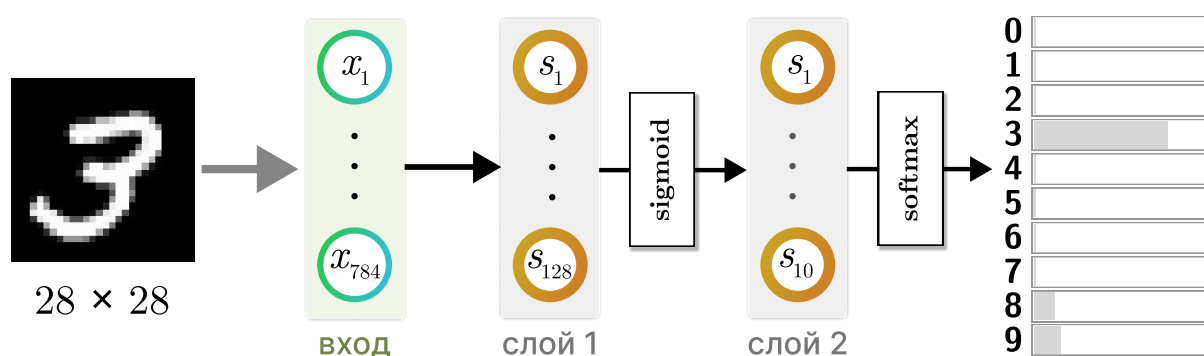


Рис. 19: Пример модели для распознавания рукописных цифр.

Первый слой модели состоит из 128 нейронов. Чем больше нейронов, тем более сложные закономерности сможет находить модель, однако большое количество нейронов делает обучение более долгим.

После обработки первым слоем применяется сигмоидальная функция активации, которую мы уже рассматривали ранее. Активации первого слоя передаются во второй слой, который содержит 10 нейронов. После преобразования особой функцией активации **softmax** мы получим набор из десяти вероятностей.

Первое число — вероятность того, что на рисунке изображена цифра 0, второе — цифра 1 и так далее до цифры 9. Как видно на

рис. 19, вероятность того, что на рисунке изображена цифра 3, наибольшая. Поскольку цифра 3 похожа на цифры 8 и 9, вероятности для них тоже ненулевые.

В этом ноутбуке вы можете самостоятельно обучить рассмотренную нами модель, протестировать её и посмотреть, как формируется распределение вероятностей для цифр.

4 Заключение

Мы увидели, что модель нейронной сети — это одно из открытий, которые были вдохновлены природой. От первоначальной модели нейронные сети долго и успешно эволюционировали и позволяют сегодня решать крайне сложные задачи.

Однако, несмотря на всю мощь искусственных нейронных сетей, они всё ещё на порядки уступают человеческому мозгу в эффективности. Чтобы компьютер по вычислительным способностям приблизился к мощности мозга человека, он должен быть способен выполнять $1.5 \cdot 10^{18}$ операций в секунду.

Если учесть, что в мозге около 10^{11} нейронов и 10^{14} синапсов, то потребуется около сотни тысяч видеокарт для моделирования умственной деятельности в мозге человека, притом что мозг потребляет около 20 Вт энергии, а ферма из тысяч видеокарт — мегаватты. Это означает, что развитие нейронных сетей не будет стоять на месте, а технологии будут совершенствоваться и становиться более эффективными.

В следующих главах мы подробно поговорим о том, как устроен процесс обучения нейронных сетей, а также о других технологиях машинного обучения и искусственного интеллекта.